PHASE 3 · INTERMEDIATE

DAY 16

Concurrency Basics

threading module · Race conditions & Locks ·
multiprocessing module · async / await · asyncio.run()

2 hours	3 concept files	26 problems	12 examples

Session Overview

Goal: Students understand Python's three concurrency models — threading for I/O-bound waiting, multiprocessing for CPU-bound parallelism, and asyncio for cooperative concurrency at scale — and know when to reach for each one.

Outcome: By the end of this session, students can spawn and join threads and processes, protect shared state with a Lock, explain why a race condition happens, and write a simple asyncio program using async/await and gather().

Time	Segment	Focus
0:00 - 0:10	Warm-up & Recap	Review Day 15 functional programming; introduce I/O vs CPU-bound work
0:10 - 0:40	Step 1	threading module: Thread, join, race conditions, Lock (File 1)
0:40 - 1:05	Step 2	multiprocessing module: Process, Queue, ProcessPoolExecutor (File 2)
1:05 - 1:15	Break	5-10 min stretch break
1:15 - 1:45	Step 3	async/await, asyncio.run(), gather() (File 3)
1:45 - 1:55	Practice Time	Students attempt Section A-C warm-up problems
1:55 - 2:00	Wrap-up	Recap, concept table, Day 17 preview

> **TIP**
>
> Concurrency is where GIL misconceptions run deepest. Be explicit: threading does NOT give true parallelism for CPU-bound Python code because of the GIL — it only helps when threads are WAITING (I/O). Multiprocessing sidesteps the GIL entirely by using separate processes. This distinction is the single most important idea today.

Prerequisites & Preparation

Requirements

Requirement	Details
Python version	3.12 (any 3.10+ works)
Editor	VS Code with Python + Pylance extensions
Prior sessions	Day 9 (Error Handling), Day 12 (Modules), Day 14 (Decorators & Closures)
Files needed	day16_01_threading.py, day16_02_multiprocessing.py, day16_03_async_await.py

Instructor Pre-Session Checklist

- Confirm all students completed Day 15 (Functional Programming) practice problems
- Open all three day16_0N_*.py concept files in VS Code, ready to run live
- Pre-run each file 2-3 times on your machine to confirm the race-condition demo reliably shows a wrong count, and the locked version reliably shows the correct count
- Remind students that multiprocessing code MUST be guarded by if __name__ == '__main__': - run day16_02 once live to show what happens without it (optional, on Windows especially)
- Prepare a simple whiteboard diagram: three lanes labelled Threading / Multiprocessing / Asyncio, each with a one-line 'best for' caption

Step-by-Step Instruction Guide

STEP 1 — The threading Module (30 min)

"Imagine you're waiting on hold with three different delivery apps at once. You're not doing three things simultaneously with your brain - you're just quickly switching attention while each call is on hold. That's exactly what threading does for I/O-bound waiting in Python."

Live-code: Starting and Joining Threads

```
import threading, time

def download_menu(restaurant_name, delay_seconds):
    time.sleep(delay_seconds)   # simulates a network wait
    print(f"Downloaded menu for {restaurant_name}")

t1 = threading.Thread(target=download_menu, args=("Punjabi Dhaba", 0.2))
t2 = threading.Thread(target=download_menu, args=("South Indian Tiffin", 0.1))
t1.start(); t2.start()      # both run concurrently from here
t1.join(); t2.join()        # wait for both to finish
```

Live-code: Race Condition, Then Fixed With a Lock

```
# UNSAFE - read/write not protected, final count is WRONG
def increment_unsafe(times):
    global unsafe_counter
    for _ in range(times):
        current = unsafe_counter
        time.sleep(0)             # yields - another thread can interleave
        unsafe_counter = current + 1
# Expected 2000, got (unsafe): 502     <- lost updates!

# SAFE - Lock makes read+write ATOMIC across threads
with counter_lock:
    current = safe_counter
    safe_counter = current + 1
# Expected 2000, got (safe): 2000      <- always correct
```

Common Error	Fix
Forgetting .join() and assuming the main program waits automatically	The main thread continues immediately after .start() - always .join() before reading results

Common Error	Fix
Believing threading gives true CPU parallelism in Python	The GIL means only one thread runs Python bytecode at a time - threading helps I/O waits, not CPU crunching
Sharing a plain list/dict across threads without a Lock	Wrap every read-modify-write of shared state in `with lock:`

DONE WHEN

Students can explain, in their own words, why the unsafe counter demo produces a different (wrong) number on different runs, and can fix it themselves using threading.Lock.

STEP 2 — The multiprocessing Module (25 min)

"Threading couldn't give us true parallelism for CPU-heavy work because of the GIL. multiprocessing solves this by launching entirely separate Python processes - each with its own interpreter and its own GIL - so they really can run at the same time on different CPU cores."

Live-code: Process + Queue for Getting Results Back

```
import multiprocessing

def compute_and_queue(number, result_queue):
    result_queue.put((number, number * number))

if __name__ == '__main__':          # REQUIRED for multiprocessing
    result_queue = multiprocessing.Queue()
    processes = [multiprocessing.Process(
        target=compute_and_queue, args=(n, result_queue)) for n in [3, 5, 7]]
    for p in processes: p.start()
    for p in processes: p.join()
# Output: 3 squared is 9 / 5 squared is 25 / 7 squared is 49
```

Live-code: ProcessPoolExecutor (Higher-Level API)

```
from concurrent.futures import ProcessPoolExecutor

with ProcessPoolExecutor(max_workers=3) as executor:
    results = list(executor.map(count_primes_up_to, [10000, 20000, 5000]))
# Output: [1229, 2262, 669]
```

Common Error	Fix
Omitting if __name__ == '__main__': around multiprocessing code	Always guard process-spawning code - required on Windows/macOS, good practice everywhere
Expecting a shared list/dict to update across processes like it would with threads	Processes do NOT share memory - use multiprocessing.Queue (or Manager) to pass data back
Using multiprocessing for I/O-bound tasks (unnecessary overhead)	Reach for threading or asyncio when the bottleneck is waiting, not computing

DONE WHEN

Students can spawn multiple processes, retrieve their results via a Queue, and explain why processes need a Queue while threads could just share a locked list.

WATCH OUT

On Windows, forgetting the __main__ guard can cause runaway recursive process spawning. Emphasize this guard as non-negotiable, not a style preference.

STEP 3 — async/await and asyncio (30 min)

"asyncio is a THIRD way to handle concurrency, and it's different from both threading and multiprocessing: everything runs on a SINGLE thread, but a coroutine can voluntarily pause at an await point and let something else run. It's cooperative, not preemptive - and it scales to thousands of concurrent tasks."

Live-code: Sequential Awaits vs Concurrent gather()

```
import asyncio

async def fetch_order_status(order_id, delay_seconds):
    await asyncio.sleep(delay_seconds)   # non-blocking wait
    return f"Order {order_id}: Delivered"

# Sequential - waits do NOT overlap (slower)
r1 = await fetch_order_status("SW1001", 0.1)
r2 = await fetch_order_status("SW1002", 0.1)   # total ~0.2s

# Concurrent - waits DO overlap (faster)
results = await asyncio.gather(
    fetch_order_status("SW2001", 0.1),
    fetch_order_status("SW2002", 0.1))   # total ~0.1s
```

Live-code: Running It All With asyncio.run()

```
async def main():
    results = await asyncio.gather(
        fetch_order_status("SW2001", 0.1),
        fetch_order_status("SW2002", 0.1),
        fetch_order_status("SW2003", 0.1))
    for r in results:
        print(r)

asyncio.run(main())     # entry point - starts the event loop
```

Common Error	Fix
Calling an async function directly and expecting it to run (e.g. fetch_order_status('X', 0.1) with no await)	Calling a coroutine function only creates a coroutine OBJECT - it must be awaited or run via asyncio.run()
Using time.sleep() instead of asyncio.sleep() inside a coroutine	time.sleep() blocks the WHOLE event loop; asyncio.sleep() only pauses that one coroutine

Common Error	Fix
Calling asyncio.run() more than once, or nesting it inside another running event loop	asyncio.run() should be called once, at the top level of the program

DONE WHEN

Students can write an async function, run it with asyncio.run(), and use asyncio.gather() to run multiple coroutines concurrently - and can explain why gather() is faster than awaiting each one sequentially.

Session Wrap-Up

Student Checklist

- I can start and join threads, and explain why threading suits I/O-bound work
- I can explain what a race condition is and fix one using threading.Lock
- I can spawn processes and retrieve results using multiprocessing.Queue
- I know why multiprocessing code must be guarded with if __name__ == '__main__':
- I can write and run a coroutine with async/await and asyncio.run()
- I can run multiple coroutines concurrently with asyncio.gather()
- I completed at least Section A-C of the Day 16 practice problems

Key Concepts Recap

Concept	One-line Summary
threading.Thread	Runs a function concurrently on a separate OS thread - great for I/O waits
Race condition	Unpredictable results when threads read/write shared state without protection
threading.Lock	Ensures only one thread executes a critical section at a time
multiprocessing.Process	Runs a function in a separate OS process, bypassing the GIL
multiprocessing.Queue	Process-safe pipe for sending results back to the parent process
ThreadPoolExecutor / ProcessPoolExecutor	Higher-level pool-based APIs for threads/processes
async def / await	Defines and pauses a coroutine cooperatively, on a single thread
asyncio.run()	Top-level entry point that starts the event loop
asyncio.gather()	Runs multiple coroutines concurrently and collects their results in order

Next Day Preview

Day 17 - Advanced OOP: Abstract base classes with ABCMeta for defining interfaces; metaclasses for controlling class creation itself; Protocol classes for structural subtyping (duck typing with type hints); and the

@dataclass decorator for concise, boilerplate-free classes.

Troubleshooting Notes

If a student's race-condition demo doesn't show a wrong count, larger iteration counts or an explicit time.sleep(0) between the read and write step forces visible interleaving - the version in day16_01_threading.py already includes this and is reliable across repeated runs. If multiprocessing code hangs or spawns endlessly on Windows, check for a missing if __name__ == '__main__': guard first.

Quick Reference Card

Daily Workflow Commands

```
cd ~/python-course/day16
uv venv                          # if not already created
source .venv/bin/activate        # macOS/Linux
.venv\Scripts\activate           # Windows
python day16_01_threading.py
python day16_02_multiprocessing.py
python day16_03_async_await.py
```

VS Code Shortcuts

Action	macOS	Windows/Linux
Run current file	Ctrl+Option+N	Ctrl+Alt+N
Toggle terminal	Ctrl+`	Ctrl+`
Format document	Shift+Option+F	Shift+Alt+F
Go to definition	Cmd+Click	Ctrl+Click

Tool Summary

Tool	Best For
threading	A FEW concurrent I/O-bound tasks (network calls, file reads)
multiprocessing	CPU-bound parallel work that needs multiple cores
asyncio	MANY concurrent I/O-bound tasks (100s-1000s), single-threaded